

OS is an interface b/w user & hardware.

→ Resource allocator

↳ RAM, CPU, I/O devices
m/m Keyboard mouse
HDD

→ OS acts as a manager → memory, process, files, security etc.

Goal

Home

Primary → convenience

Secondary → efficiency

main frame, dev

Primary → efficiency

Secondary → convenience

Types of OS

① Batch OS :

→ whoever gives the job first should finish it, then only other jobs will be taken for processing.

J_1, J_2, J_3, J_4
↓ ↓
I/O 10 sec
100 sec CPU

$$J_1 = 100 + 10 = 110 \text{ sec}$$

$J_1 \rightarrow$ CPU 10 sec
I/O → 10 hr

CPU idle

→ J_2 can't process until J_1 if finished CPU + I/O

→ Efficiency is less, since starvation.

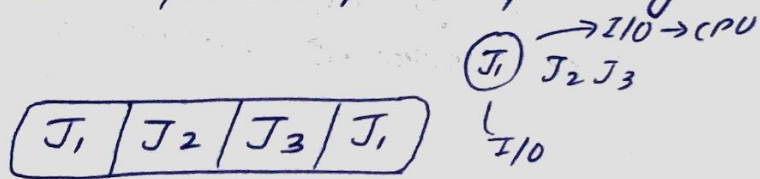
→ Users do not interact with computer directly instead each user enters the program on devices like punch cards, then it submitted to operator. acc. to requirement & executed completely in similar kind of group. Output also in punchcards. eg: bulk salary credit system.

② Multi Programming OS

- An extension (modification) of batch OS
- While J_1 is in I/O, CPU is idle & now we can allow J_2 to execute on CPU.
- Efficiency increased.

③ Multi-tasking or Time Sharing OS:-

- Multi programming OS with preemptive facility



- ① Preemptive Jobs (Snatch CPU resources before job is finished)
- ② Non-preemptive Jobs

- CPU share all jobs for some time without completing one or other. In this way interactivity can be achieved.

④ Multi-processing OS

- Instead of one CPU we have multiple CPUs.
- Jobs can be executed on all CPUs simultaneously.
- Efficiency increased.
- Throughput is improved.
 - ↳ no. of processes executed / unit time.

⑤ Real time OS

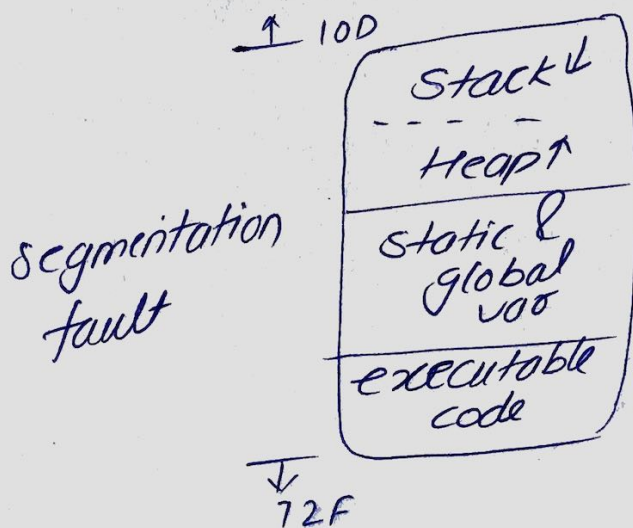
- Jobs will be given with deadlines. ($T_1 \rightarrow T_1$)
- Jobs must be finished within deadlines.

CPU Bound Process → more CPU time req.

I/O Bound Process → more I/O time req.

Process management

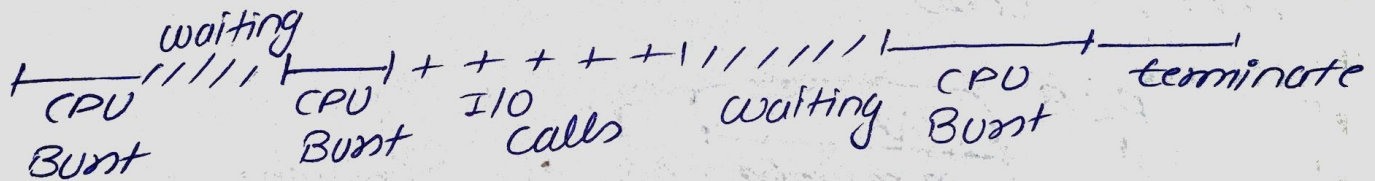
- ① Program → code (HDD)
- ② Process → when OS takes this program & (RAM) load into mem (RAM) for execution.



Attributes of a Process :-

- ① Process ID :- unique id no.s
- ② Program counter → contains address of next instr
- ③ Process state
- ④ Priority
- ⑤ General purpose registers
- ⑥ list of open files
- ⑦ list of devices
 ↳ biometric
- ⑧ Protection

Process life cycle :

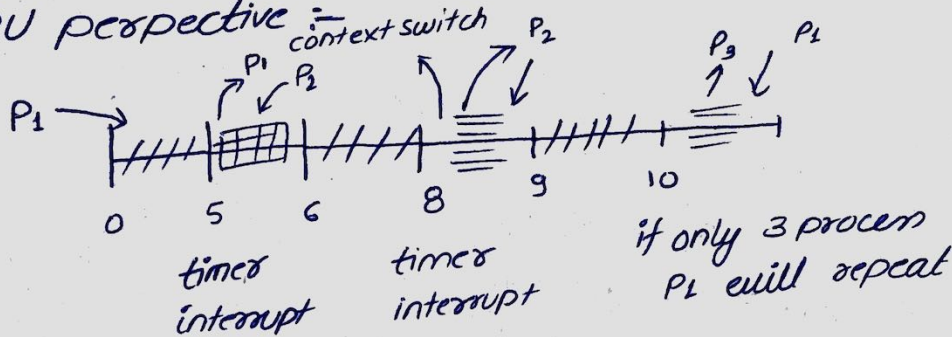


Operating System

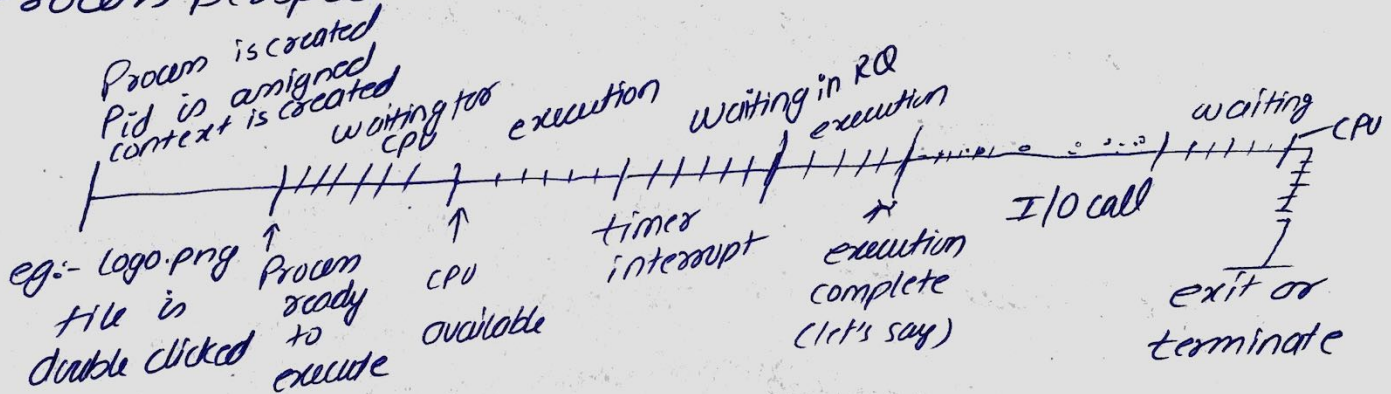
Program: Set of instructions stored in a disk

Process: Program under execution

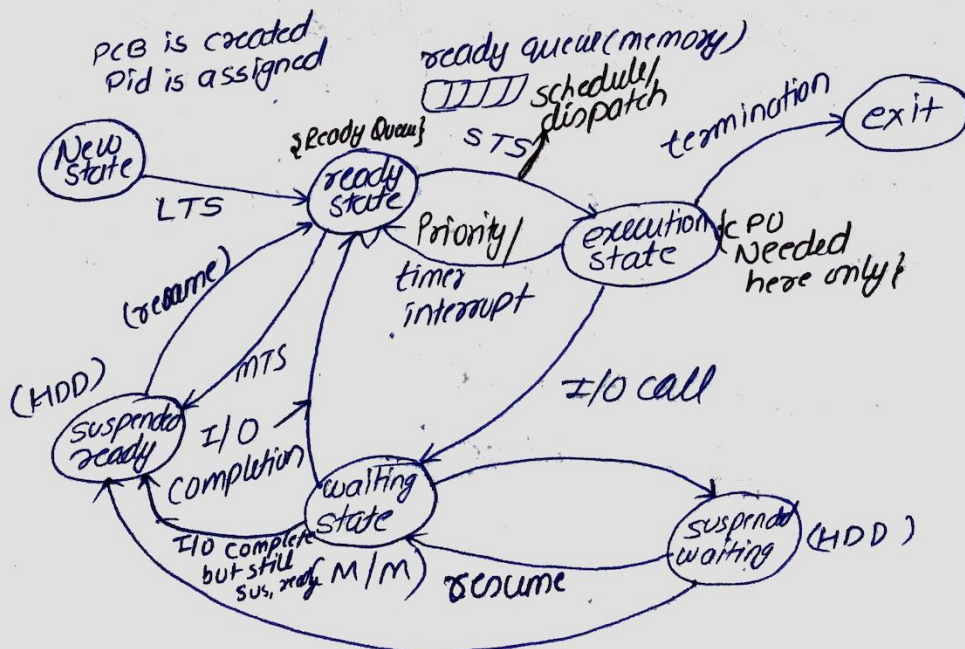
• CPU perspective :-



• Process perspective



• O.S. Perspective :- (5-state or 7-state Diagram)



(min no. of states a process needs to travel for completion - 4)

* If the ready state queue is full then the extra process will go to suspended ready state until there is space in queue again (same for suspended waiting state)

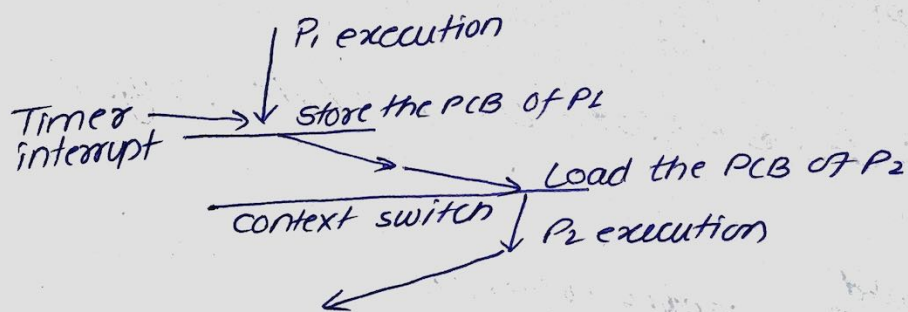
Scheduler:

Use to select which process will go into m/m first, ^{execution state}

- 3 types :-
- 1) Short-Term (STS)
 - 2) medium-Term (MTS)
 - 3) Long-Term (LTS)

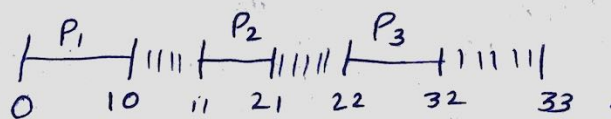
Dispatcher: It will take the process into execution state.

Context Switch:



$$T = 10 \text{ msec}$$

$$T_{\text{CS}} = 1 \text{ msec}$$



$$\text{Total Time} = 33 \text{ msec}$$

$$\text{Utilised Time} = 30 \text{ msec}$$

$$\text{CPU \%} = \frac{\text{Utilised time}}{\text{Total time}} \times 100$$

$$= \frac{30}{33} \times 100 = 90.9\%$$

8. If context switch time is half of the timer interrupt time.

Case I: Let $T = x$ Let there be n processes

$$T_{\text{cst}} = x/2$$

$$\text{Then CPU\%} = \frac{x \times n}{(x + x/2) \times n} \times 100$$

$$= \frac{2x}{3x} \times 100 = 66.66\%$$

Case II: If $T = x$ and $T_{\text{cst}} = x$ then $\text{CPU\%} = \frac{n \times x}{(x + x) \times n} \times 100 = 50\%$

Case III: If $T_{\text{cst}} = 0$ then $\text{CPU\%} = 100\%$

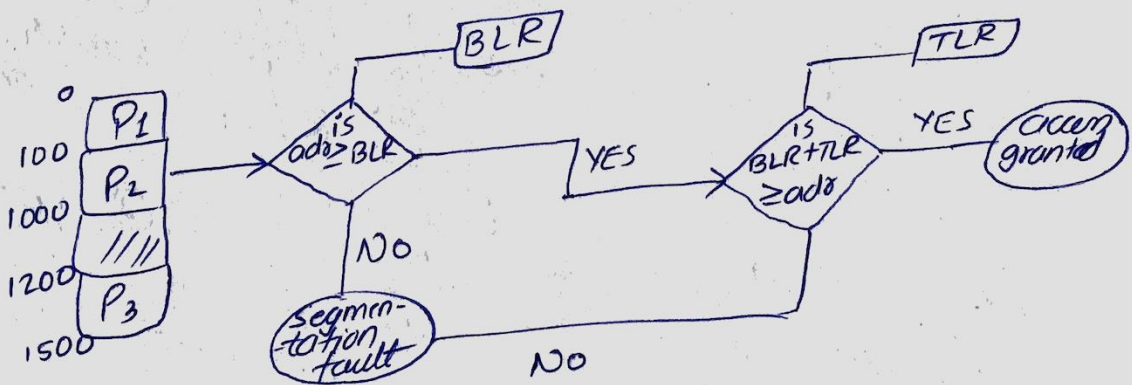
Case IV If $T = 0$ then $\text{CPU\%} = 0\%$. Known as Thrashing

Protection Provided to Hardware by an OS.

1) Protection is provided to memory by using 2 registers :- Base Length Register, Total Limit Register (BLR) (TLR)

Let's say
 P_1 is
executing
 $\text{BLR} = 0$
 $\text{TLR} = 100$

gives
segmentation
fault for
200



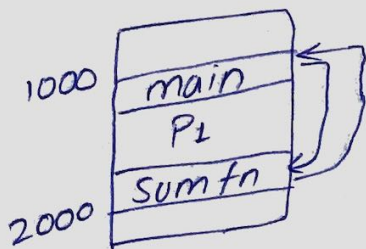
If P_2 is executing for 200 then $\text{BLR} = 100$ and

total range of limit $\rightarrow \text{TLR} = 900$ and gives access granted

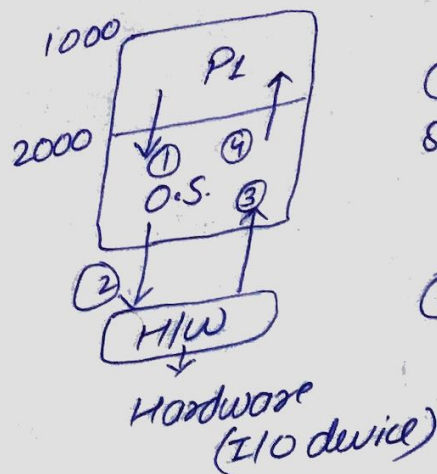
- 2) Protection to CPU is provided by O.S. using a timer device (timer interrupt done at back)
- 3) Protection to I/O devices is provided by O.S. using system calls.

System calls:-

- System calls are the service provided by an OS in form of functions to access I/O devices.
- It generates software interrupts.
- O.S. will check the privileges of the user before executing system call.
- System calls are slower than function calls because it will result in 2 context switches.



no context switch in fn call as it is not going outside



① & ④ context switch (it happens only b/w programs)

② System call

Categories of system calls

a) Process related

eg:- `fork()` :- To create new process { return pid of newly created process }
`exit()` :- Terminate normally
`kill()` :- Terminate abnormally

b) Memory related

→ swap space accessing
→ shared m/m accessing

c) I/O related :-

→ read, write, open device, seek

d) Disk related

format, partition, fragment, defragment, Quota

FAT → File Allocation Table
NTFS → New Table File System

e) Information related

`getpid()`, `setpid()`, `gettime()`, `settime()`

Dual mode Operation :-

① Supervisor mode

or
kernel mode

or
monitor mode

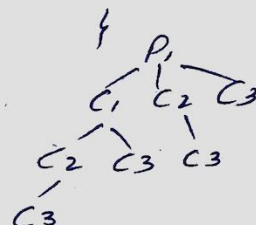
② User mode

→ `$ mysql -u root -P ↓`

`$ sudo mysql -u root -P ↓`

→ mode bit register is available on H/W who identifies which mode is running.

```
int main() {  
    fork();  
    fork();  
    fork();  
}
```



Total no. of processes = 2^n

Total no. of child processes = $2^n - 1$

User mode MBR = 0
Kernel mode MBR = 1

Instructions are of two types:-

1) Privileged instruction 2) Unprivileged instruction

→ Privileged instruction can only be executed in kernel mode.

→ O.S. checks the value of MBR before executing any privileged instruction, if it is not kernel mode then permission is denied.

eg: Switch off all interrupts → privileged instruction
cd → change directory → unprivileged instruction

→ All system calls are privileged instruction.

Command Interpreter (CI):-

Bootstrap loader:-

It is a special program stored on ROM which is used to load part of O.S. called kernel from the I/O device.

Kernel:-

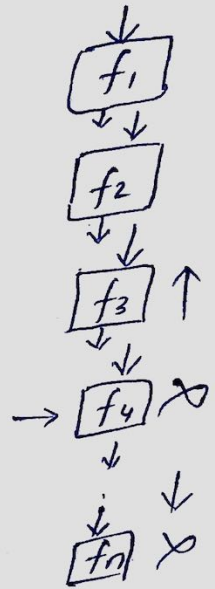
The essential part of O.S. that always resides in memory & is essential for operating of the system.

Microkernel:

- Push some utilities from O.S. to user space & run them as user level processes.
- These utilities can be invoked using message passing technique through kernel.

Layered approach to O.S. design:-

- layered approach is used in O.S. by which we can modify any layer at any time by just keeping its i/p & o/p intact.
- It is easy to debug
- It is slow but more robust to crash.
- Difficult to figure out what module should go in which layer.



Process Scheduling

- ① FCFS
- ② SJFS $\begin{cases} \text{Preemptive} \\ \text{non-preemptive} \end{cases}$
- ③ LJFS $\begin{cases} P \\ NP \end{cases}$
- ④ Priority $\begin{cases} P \\ NP \end{cases}$
- ⑤ Round Robin
- ⑥ HRRN (Highest Response Ratio Next)

Criteria to judge which solⁿ algo is good/better.

① Response time :- Time taken by a process to get into CPU for first time.

$$\text{Response time} = \left| \text{Time of arrival} - \text{Time at which CPU is allocated} \right|$$

② Waiting time :- Total time a process has to wait in Ready Queue + waiting Queue.

③ Turnaround time :- Total time spent by a process in the system.

$$\text{Turnaround time} = \left| \text{Time of completion} - \text{Time of arrival} \right|$$

* Burst time : Total time a process wants to spend in CPU.

④ CPU utilization : Amount of % a CPU is kept busy doing useful task.

$$\text{CPU \%} = \frac{\text{useful time}}{\text{total time}} \times 100$$

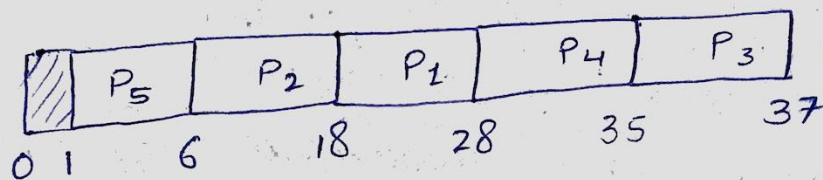
⑤ Throughput : No. of processes running per unit time.

$$\text{Throughput} = \frac{\text{No. of processes}}{\text{total time}}$$

⑥ Time at which a process completes.

Q. Process Burst time Arrival time

P_1	10	3
P_2	12	2
P_3	2	7
P_4	7	4
P_5	5	1



↑
Gantt's chart

Process	Response time	Waiting time	Completion time	Turnaround time
P_1	15	15	28	25
P_2	4	4	18	16
P_3	28	28	37	30
P_4	24	24	35	31
P_5	0	0	6	5
	<u>71</u>			<u>107</u>

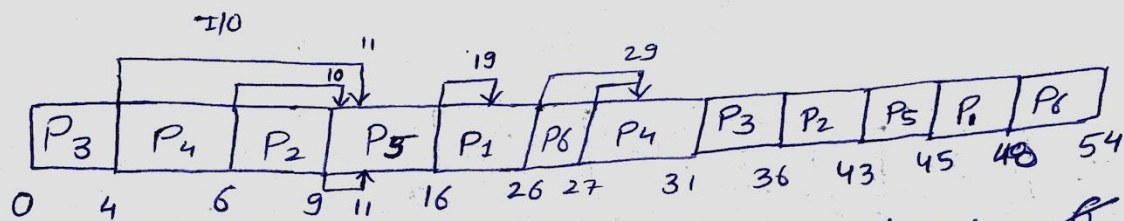
$$\text{Avg. RT} = \frac{71}{5}$$

$$\text{Avg. TAT} = \frac{107}{5}$$

$$\text{CPU \%} = \frac{36}{37} \times 100$$

$$\text{Throughput} = \frac{5}{37}$$

8. Process	Burst Time	I/O time	Arrival time	Start time	Completion time	TAT	Waiting time	Res. time
P ₁	10, 3	3	4:29	16	48	44	28	12
P ₂	3, 7	2	2:11	6	43	41	29	4
P ₃	4, 5	7	0:11	0	36	36	20	0
P ₄	2, 4	4	1:10	4	31	30	20	3
P ₅	3, 2	3	3:19	9	45	42	30	6
P ₆	1, 6	2	5:29	26	54	49	40	21
						242	167	46
						Avg = 40.33 Avg = 27.83 Avg = 7.67		

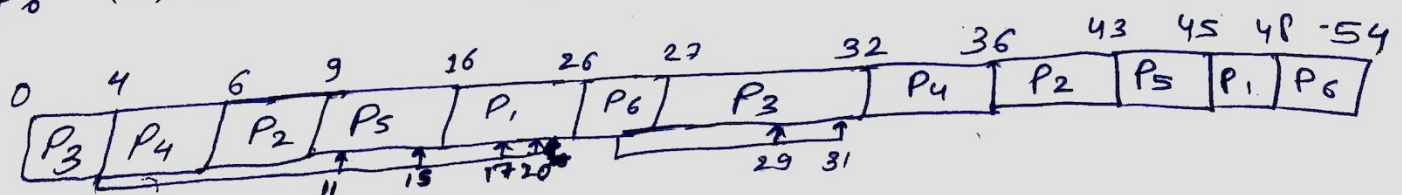


$$CPU\% = \frac{54}{54} \times 100 = 100\% \quad \text{Throughput} = \frac{6}{54} = \frac{1}{9}$$

$$\text{Waiting time} = \text{turnaround time} - (\text{total burst time} + \text{I/O time})$$

* Convoy effect is the disadvantage of FCFS scheduling algo.

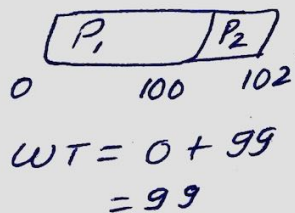
	Arrival time	Comp time	TAT	WT	RT
P ₁	4:29	48	44	28	12
P ₂	2:17	43	41	29	4
P ₃	0:11	36	36	16	0
P ₄	1:15	39	35	25	3
P ₅	3:20	45	42	30	6
P ₆	5:31	54	49	40	21



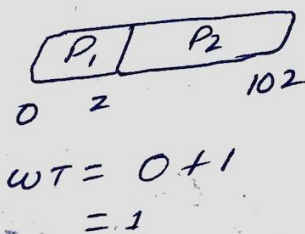
* Convoy effect

FCFS suffers from convoy effect.

	AT	BT
P ₁	0	100
P ₂	1	2

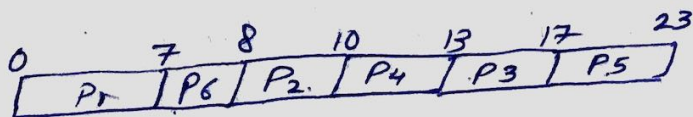


	AT	BT
P ₁	0	2
P ₂	1	100



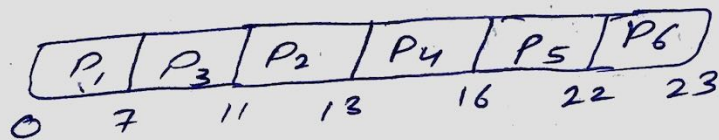
Shortest Job First Scheduling (SJFS) {Non-preemptive}

Process	BT	AT	ST	CT	TAT	WT	RT
P ₁	3	7	0	7	7	0	0
P ₂	4	2	3	10	7	5	5
P ₃	2	4	2	13	15	11	11
P ₄	5	3	4	10	9	6	6
P ₅	3	6	5	17	18	12	12
P ₆	2	1	7	8	1	0	0
Avg					9.5	5.67	5.67



FCFS

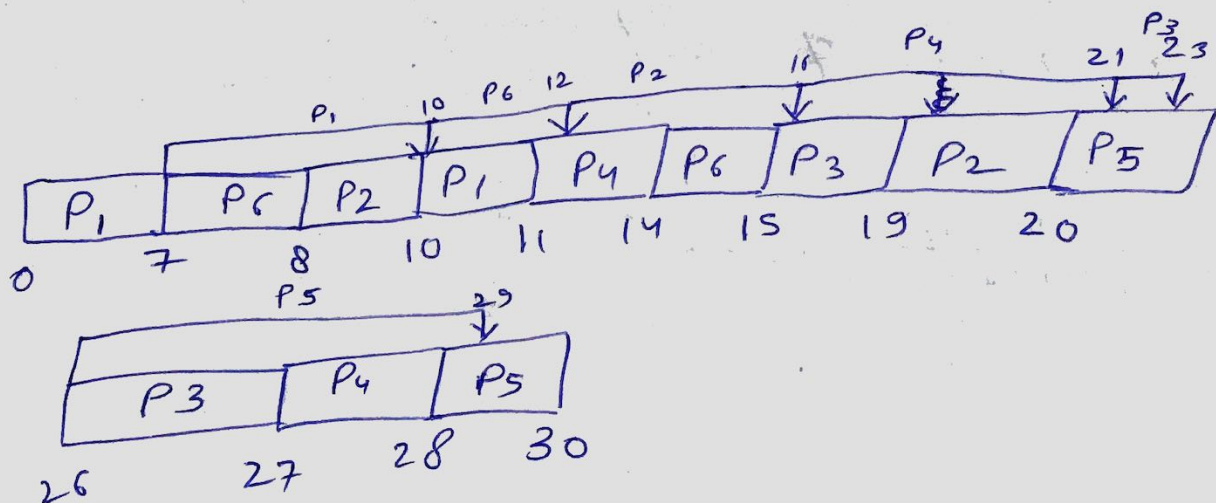
Process	B/T	A/T	S/T	C/T	TAT	W/T	P/T
P ₁	7	0	0	7	7	0	0
P ₂	2	3	8	13	10	8	8
P ₃	4	3	7	11	9	5	5
P ₄	3	4	13	16	12	9	9
P ₅	6	5	16	22	17	11	11
P ₆	1	7	22	23	16	15	15
Avg					11.8	8	8



One I/O & with multiple I/O device

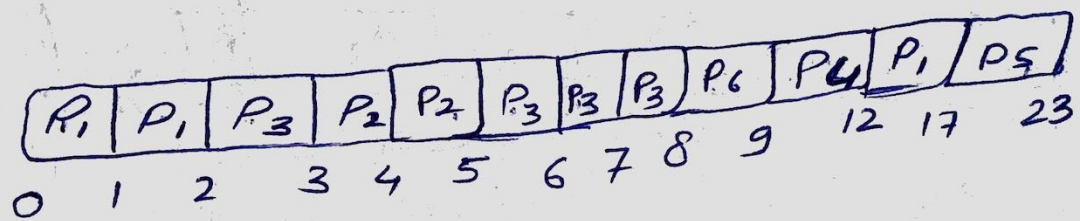
i) With one I/O

Process	BT	AT	I/O	ST	CT	RT	TAT	TBT	WT
P ₁	7,1	0 10	3	0	11	0	11	11	0
P ₂	2,1	3 16	4	8	20	5	17	7	10
P ₃	4,1	2 23	2	15	27	13	25	7	18
P ₄	3,1	4 21	5	11	28	7	24	9	15
P ₅	6,1	5 29	3	20	30	15	25	10	15
P ₆	1,1	7 12	2	27	15	0	8	4	4
						6.67	18.33		10.33



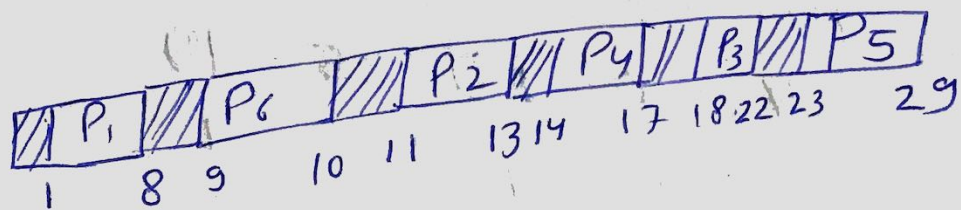
Shortest Remaining time first (SRTF)
or SJFS with preemption

Process	$\frac{B}{T}$	$\frac{A}{T}$	$\frac{S}{T}$	$\frac{C}{T}$	TAT	$\frac{W}{T}$	$\frac{R}{T}$
P_1	7/5	0	0	17	17	10	0
P_2	2/10	3	3	5	2	0	0
P_3	4/3 10/2	2	2	8	6	2	0
P_4	3/7 10/4	9	12	8	5	5	
P_5	6	5	17	23	18	12	12
P_6	1	7	8	9	2	1	1



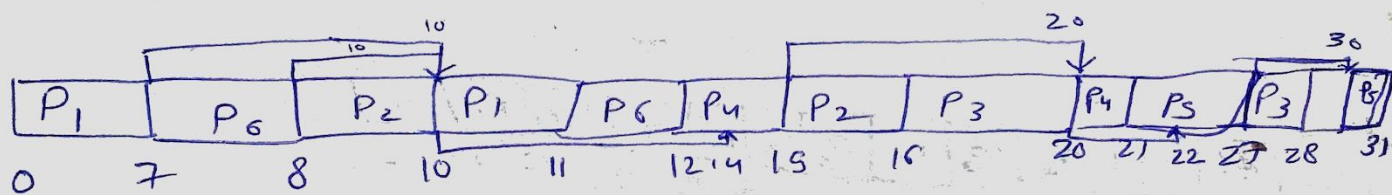
i) SJFS with $CST = 1ms$

Process	BT	AT	ST	CT	RT	TAT	WT
P_1	7	0	1	8	1	8	1
P_2	2	3	11	13	8	10	8
P_3	4	2	18	22	16	20	16
P_4	3	4	14	17	10	13	10
P_5	6	5	23	29	18	24	18
P_6	1	7	9	10	2	3	2



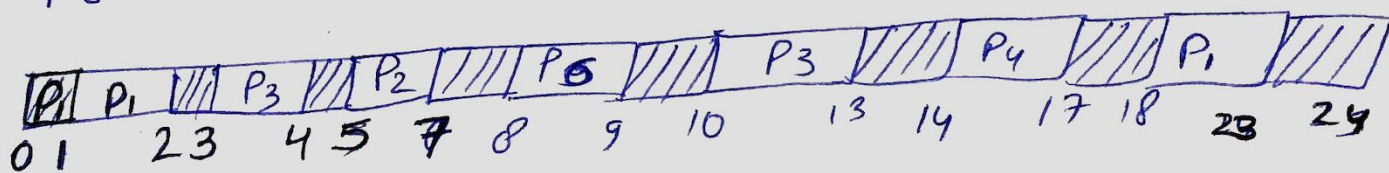
i) multiple IO device SJFS

Process	BT	AT	IO	ST	CT	RT	TAT	TBT	WT
P ₁	7,1	010	3	0	11	0	11	11	0
P ₂	2,1	014	4	8	16	5	13	7	6
P ₃	4,1	022	2	16	28	14	26	7	19
P ₄	3,1	020	5	12	21	8	17	9	8
P ₅	6,1	030	3	21	31	16	26	10	16
P ₆	1,1	010	2	7	12	0	5	4	1
						<u>7.16</u>	<u>16.33</u>		<u>8.33</u>



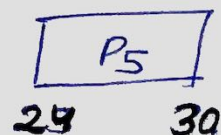
ii) SRTF with CST = 1ms

Process	BT	AT	ST	CT	RT	TAT	WT
P ₁	7	0	1	23	0	23	16
P ₂	2	3	5	7	2	4	2
P ₃	4	2	3	13	1	11	7
P ₄	3	4	14	17	10	13	10
P ₅	6	5	24	30	29	25	19
P ₆	1	7	8	9	1	2	1



$$\text{CPU\%} = \frac{23}{30} \times 100$$

$$\text{throughput} = \frac{6}{30}$$



FCFS program

```
int main(){
    int n;
    cout << "Enter no. of processes: ";
    cin >> n;
    vector<int> pid(n), at(n), bt(n), ct(n), st(n), tct(n),
        wct(n), rt(n);

    for(int i=0; i<n; i++){
        cout << "Enter pid, arrival time and burst
            time of process" << i+1 << " : ";

        cin >> pid[i];
        cin >> at[i];
        cin >> bt[i];
    }

    // sort by arrival time and by pid if arrival
    // time is same
    for(int i=0; i<n; i++){
        for(int j=0; j<n-i-1; j++){
            if (at[i] > at[j+1] || (at[i] == at[j+1] &&
                pid[i] > pid[j+1])){
                swap(at[i], at[j+1]);
                swap(bt[i], bt[j+1]);
                swap(pid[i], pid[j+1]);
            }
        }
    }
}
```

// calculate start time, completion time, turnaround time, waiting time, and response time

$st[0] = at[0];$

$ct[0] = st[0] + bt[0];$

$tat[0] = ct[0] - at[0];$

$wt[0] = tat[0] - bt[0];$

$rt[0] = st[0] - at[0];$

```
for (int i=1; i<n; i++){  
     $st[i] = \max(ct[i-1], at[i]);$   
     $ct[i] = st[i] + bt[i];$   
     $tat[i] = ct[i] - at[i];$   
     $wt[i] = tat[i] - bt[i];$   
     $rt[i] = st[i] - at[i];$   
}
```

// output

$cout << "PID \backslash Arrival Time \backslash Burst Time \backslash start$
 $time \backslash Completion Time \backslash Turnaround time \backslash$
 $waiting time \backslash Response time \backslash n";$

```
for (int i=0; i<n; i++){  
     $cout << pid[i] << at[i] << bt[i] << st[i] <<$   
 $ct[i] << tat[i] << wt[i] << rt[i];$   
}
```

// order

```
for (int i=0; i<n; i++){  
     $cout << "P" << pid[i];$   
    if (i < n-1)  $cout << " \rightarrow ";$   
}
```

```
double avgTat = accumulate(tat.begin(), tat.end(), 0.0) / n;  
double avgwt = accumulate(wt.begin(), wt.end(), 0.0) / n;  
double avgrt = accumulate(rt.begin(), rt.end(), 0.0) / n;
```

```
cout << "In Average Turnaround time:" << avgTat;
```

```
cout << "In Average Waiting time:" << avgwt;
```

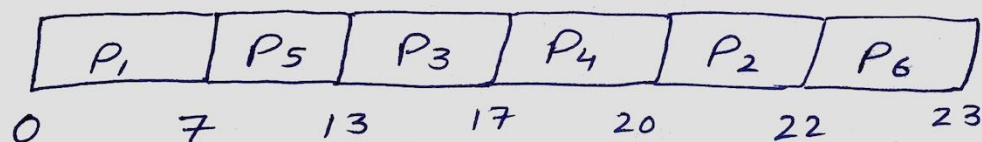
```
cout << "In Average Response time:" << avgrt;
```

```
return 0;
```

```
{
```

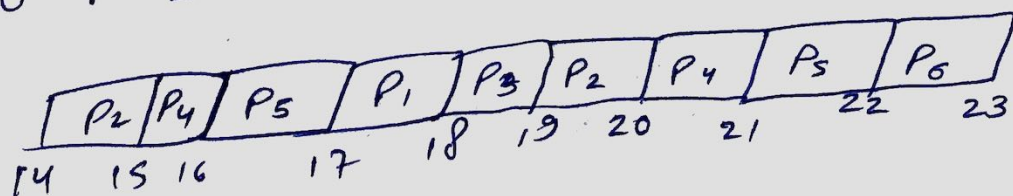
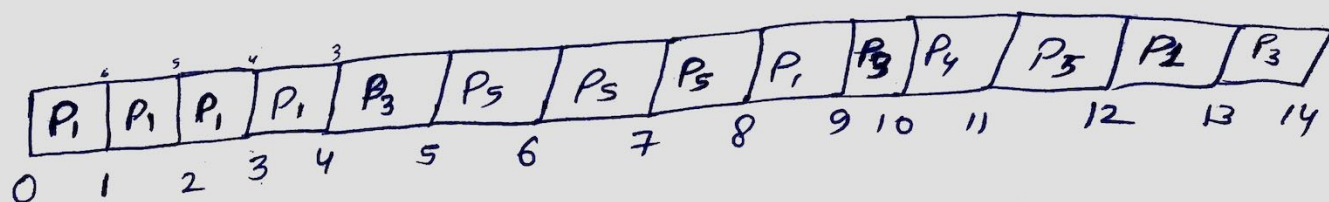
Longest Job first Scheduling

Process	BT	AT	ST	CT	RT	TAT	WT
P ₁	7	0	0	7	0	7	0
P ₂	2	3	20	22	17	19	17
P ₃	4	2	13	17	11	15	11
P ₄	3	4	17	20	13	16	13
P ₅	6	5	7	13	2	8	2
P ₆	1	7	22	23	15	16	15



Longest Remaining time first

Process	BT	AT	ST	CT	RT	TAT	WT
P ₁	7 2	6 3	0	18	0	18	16
P ₂	2	1	3	14	11	17	15
P ₃	4	2	2	4	19	2	17
P ₄	3	2	4	10	6	17	14
P ₅	6	5	5	21	0	17	11
P ₆	1	2	7	22	23	15	15



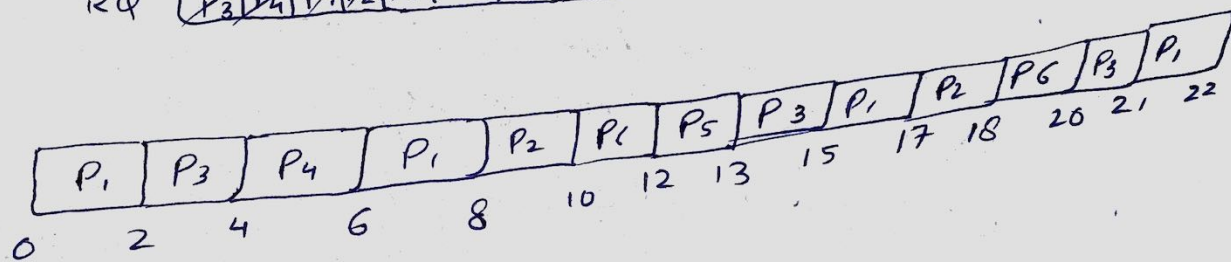
Round Robin Scheduling:-

t_q (time Quanta)

$t_q = 2ms$

Process	Burst time	Arrival time
P_1	7	0
P_2	3	3
P_3	5	1
P_4	2	2
P_5	1	4
P_6	4	3

RQ $\boxed{P_3} \boxed{P_4} \boxed{P_1} \boxed{P_2} \boxed{P_6} \boxed{P_5} \boxed{P_3} \boxed{P_1} \boxed{P_2} \boxed{P_6} \boxed{P_3} \boxed{P_1}$

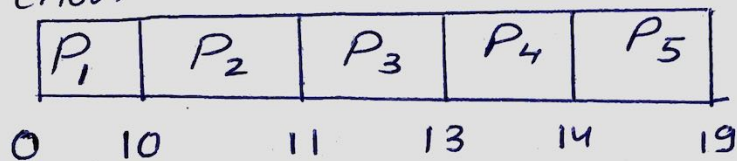


SIFS } Program
LJFS }

Q1. FCFS

Process	$\frac{B}{T}$	$\frac{A}{T}$	$\frac{S}{T}$	$\frac{C}{T}$	$\frac{R}{T}$	$\frac{T}{T}$	$\frac{W}{T}$
P ₁	10	0	0	10	0	10	0
P ₂	1	0	10	11	10	11	10
P ₃	2	0	11	13	11	13	11
P ₄	1	0	13	14	13	14	13
P ₅	5	0	14	19	14	19	14
				<u>9.6</u>	<u>13.4</u>	<u>9.6</u>	<u>Avg</u>

Gantt chart



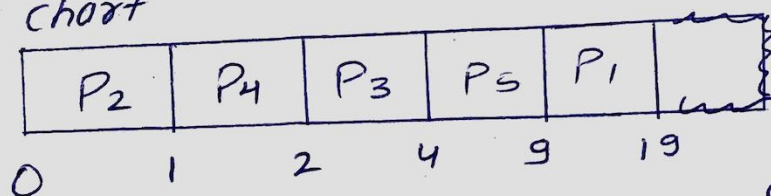
$$CPU\% = \frac{19 \times 100}{19} = 100\%$$

$$Throughput = \frac{5}{19}$$

2. SJFS (FCFS for same burst time)

Process	$\frac{B}{T}$	$\frac{A}{T}$	$\frac{S}{T}$	$\frac{C}{T}$	$\frac{R}{T}$	$\frac{T}{T}$	$\frac{W}{T}$
P ₁	10	0	9	19	9	19	9
P ₂	1	0	0	1	0	1	0
P ₃	2	0	2	4	2	4	2
P ₄	1	0	1	2	1	2	1
P ₅	5	0	4	9	4	9	4
				<u>3.2</u>	<u>7</u>	<u>3.2</u>	<u>Avg</u>

Gantt chart



$$CPU\% = \frac{19 \times 100}{19} = 100\%$$

$$= 100\%$$

$$Throughput = \frac{5}{19}$$

3. Round Robin (Quantum = 1)

(ST-AT) (CT-AT) (TAT-BT)

Process	$\frac{B}{T}$	$\frac{A}{T}$	$\frac{S}{T}$	$\frac{C}{T}$	$\frac{R}{T}$	$\frac{T}{T}$	$\frac{W}{T}$
P ₁	10	0	0	19	0	19	9
P ₂	1	0	1	2	1	2	1
P ₃	2	0	2	7	2	7	5
P ₄	1	0	3	4	3	4	3
P ₅	5	0	4	14	4	14	9
	<u>2</u>			<u>9.2</u>		<u>5.4</u>	

$$\text{Throughput} = \frac{5}{19}$$

Avg.

Gantt chart

$$\text{CPU\%} = \frac{19}{19} \times 100 = 100\%$$

P_1	P_2	P_3	P_4	P_5	P_1	P_3	P_5	P_1	P_5	P_1	P_5	P_1	P_5	P_1	P_4	P_1
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

4. SRTF

Process	$\frac{A}{T}$	$\frac{B}{T}$	$\frac{S}{T}$	$\frac{C}{T}$	$\frac{R}{T}$	$\frac{T}{T}$	$\frac{W}{T}$
P ₁	0	8	0	17	0	17	9
P ₂	1	4	1	5	0	4	0
P ₃	2	9	17	26	15	24	15
P ₄	3	5	5	10	2	7	2
						<u>13</u>	<u>6.5</u>

Avg.

Gantt chart

P_1	P_2	P_2	P_2	P_2	P_4	P_1	P_3	
0	1	2	3	4	5	10	17	26

$$\text{CPU\%} = \frac{26}{26} \times 100$$

$$= 100\%$$

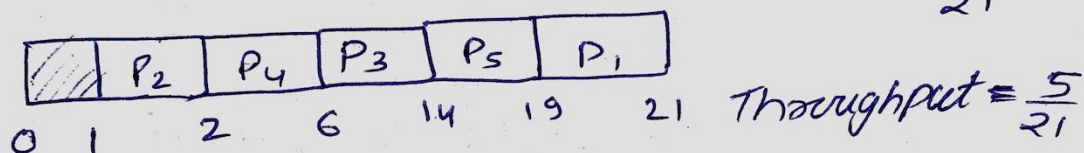
$$\text{Throughput} = \frac{4}{26}$$

5. LJFS

Process	$\frac{B}{T}$	$\frac{A}{T}$	$\frac{S}{T}$	$\frac{C}{T}$	$\frac{R}{T}$	$\frac{T}{A}$	$\frac{W}{T}$
P_1	2	2	19	21	17	19	17
P_2	1	1	1	2	0	1	0
P_3	8	4	6	14	2	10	2
P_4	4	2	2	6	0	4	0
P_5	5	3	14	19	$\frac{11}{6}$	$\frac{16}{10}$	$\frac{11}{6}$
AVG							

Gantt chart

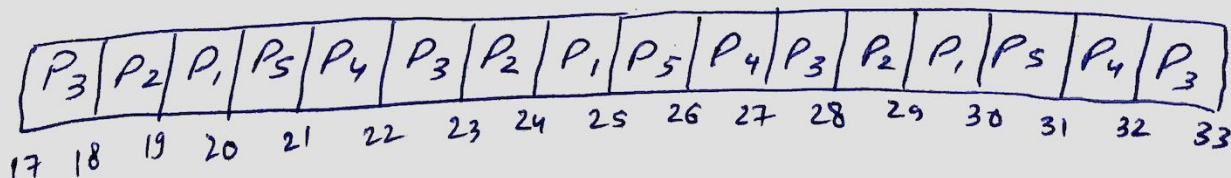
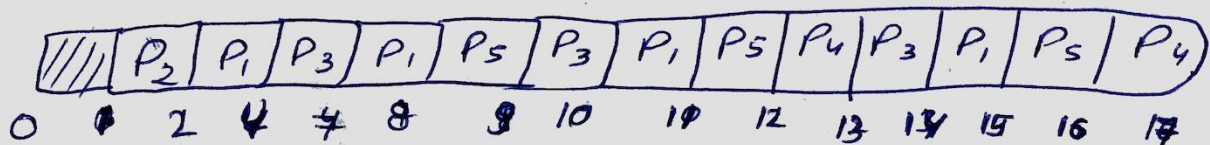
$$CPU\% = \frac{20}{21} \times 100$$



6. LRTF

Process	$\frac{B}{T}$	$\frac{A}{T}$	$\frac{S}{T}$	$\frac{C}{T}$	$\frac{R}{T}$	$\frac{T}{A}$	$\frac{W}{T}$
P_1	8	2	2	30	0	28	20
P_2	4	1	1	29	0	28	24
P_3	9	4	4	33	0	29	20
P_4	5	3	12	32	9	29	24
P_5	6	2	8	31	$\frac{6}{3}$	$\frac{29}{27.7}$	$\frac{23}{22.2}$
AVG							

$$CPU\% = \frac{32}{33} \times 100$$



SJFS program

```
int main() {
    int n;
    cout << "Enter no. of process ";
    cin >> n;
    vector<int> pid(n), at(n), bt(n), ct(n), st(n), tot(n),
        wt(n), rt(n), indices(n);

    for (int i = 0; i < n; i++) {
        cout << "Enter pid, arrival time & burst ";
        cin >> pid[i] >> at[i] >> bt[i];
        indices[i] = i;
    }
    sort(indices.begin(), indices.end(), [&](int a, int b) {
        return at[a] < at[b] || (at[a] == at[b] && a < b);
    });
    vector<bool> completed(n, false);
    int t = 0;
    int pr = 0;
    while (pr < n) {
        int mni = -1;
        int mibt = INT_MAX;
        for (int i = 0; i < n; i++) {
            int idx = indices[i];
            if (at[idx] <= t && !completed[idx] &&
                bt[idx] < mibt) {
                mibt = bt[idx];
                mni = idx;
            }
        }
        t += mibt;
        completed[mni] = true;
        pr = mni + 1;
    }
}
```

```
if (mni == -1) {
```

```
    t++;  
    continue;  
}
```

```
    st[mhi] = t;
```

```
    ct[mhi] = st[mhi] + bt[mhi];
```

```
    tat[mhi] = ct[mni] - at[mni];
```

```
    wt[mhi] = tat[mni] - bt[mni];
```

```
    rt[mhi] = st[mni] - at[mni];
```

```
    t = ct[mni];
```

```
    completed[mni] = true;
```

```
    pr++;
```

```
}
```

```
// output
```

```
cout << "In PID \t Arrival \t Burst \t Start \t completion  
    \t Turnaround \t Response \t waiting \n";
```

```
for (int i = 0; i < n; i++) {
```

```
    cout << pid[i] << at[i] << bt[i] << st[i]  
    << ct[i] << tat[i] << rt[i] << wt[i];
```

```
}
```

```
double avgat = accumulate(tat.begin(), tat.end(),  
    0.0) / n;
```

```
double avgwt = accumulate(wt.begin(), wt.end(), 0.0) / n;
```

```
double avgrt = accumulate(rt.begin(), rt.end(), 0.0) / n;
```

```
cout << avgat << avgwt << avgrt;
```

```
}
```

LJFS code

```
int main(){
```

```
    int n;
```

```
    cin >> n;
```

```
    vector<int> pid(n), at(n), bt(n), ct(n), st(n),  
                tat(n), wt(n), rt(n), idx(n);
```

```
    for(int i=0; i<n; i++){  
        cin >> pid[i] >> at[i] >> bt[i];  
        idx[i] = i;
```

```
    }
```

```
    sort(idx.begin(), idx.end(), [&](int a, int b){  
        return at[a] < at[b] || (at[a] == at[b] &&  
            a < b);
```

```
});
```

```
vector<bool> completed(n, false);
```

```
int t=0;
```

```
int pr=0;
```

```
while(pr < n){
```

```
    int mbi = -1;
```

```
    int mbt = INT_MIN;
```

```
    for(int i=0; i<n; i++){
```

```
        int ind = idx[i];
```

```
        if(at[ind] <= t && !completed[ind] &&
```

```
            bt[ind] > mbt){
```

```
            mbt = bt[ind];
```

```
            mbi = ind;
```

```
        }  
    }
```

```
if (mbi == -1) {  
    t++;  
    continue;  
}
```

```
st[mbi] = t;  
ct[mbi] = st[mbi] + bt[mbi];  
tat[mbi] = ct[mbi] - at[mbi];  
wt[mbi] = tat[mbi] - bt[mbi];  
rt[mbi] = st[mbi] - at[mbi];
```

```
t = ct[mbi];  
completed[mbi] = true;  
pr++;
```

```
}
```

```
for (int i = 0; i < n; i++) {  
    cout << pid[i] << at[i] << bt[i] << st[i] <<  
    ct[i] << tat[i] << wt[i] << rt[i];
```

```
}
```

```
double atat = accumulate(tat.begin(), tat.end(), 0.0) / n;  
double awt = accumulate(wt.begin(), wt.end(), 0.0) / n;  
double art = accumulate(rt.begin(), rt.end(), 0.0) / n;
```

```
cout << atat << awt << art;
```

```
return 0;
```

```
}
```

Highest Response Ratio Next (HRRN)

$$\text{Response ratio} = \frac{w+s}{s}$$

where, w = waiting time at time t
 s = burst time of process p .

Process	AT	BT	ST	CT
P_0	0	4	0	4
P_1	3	8	4	12
P_2	6	6	12	18
P_3	9	7	21	30
P_4	12	3	18	21

P_0	P_1	P_2	P_4	P_3	
0	4	12	18	21	30

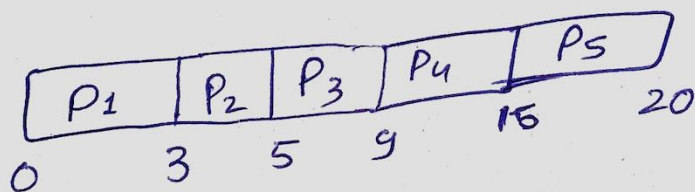
$$\begin{aligned}
 t=2 \quad RR_2 &= \frac{6+6}{6} = 2 \\
 RR_3 &= \frac{3+7}{7} = 1.4 \\
 RR_4 &= \frac{0+3}{3} = 1
 \end{aligned}$$

$$t=18 \quad RR_3 = \frac{9+7}{7} = \frac{16}{7} = 2.2$$

$$RR_4 = \frac{3+6}{3} = 3$$

→ HRRN not only favours shorter jobs but also limits the waiting time of longer jobs.

Process	BT	AT	ST	CT	(T-AT) TA	(TAT-BT) WT
P ₁	3	0	0	3	3	0
P ₂	2	1	3	5	4	2
P ₃	4	2	5	9	7	3
P ₄	6	3	9	15	12	6
P ₅	5	4	15	20	16	11



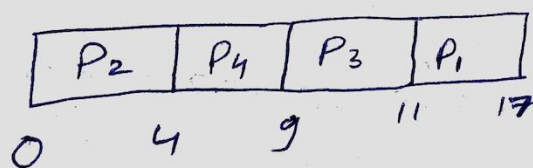
$$\begin{aligned}
 t=3 \quad RR_2 &= \frac{2+2}{2} = 2 \\
 RR_3 &= \frac{1+4}{4} = 1.25 \\
 RR_4 &= \frac{0+6}{6} = 1
 \end{aligned}$$

$$\begin{aligned}
 t=5 \quad RR_3 &= \frac{3+4}{4} = 1.8 \\
 RR_4 &= \frac{2+6}{6} = \frac{8}{6} = 1.5 \\
 RR_5 &= \frac{1+5}{5} = \frac{6}{5} = 1.2
 \end{aligned}$$

$$\begin{aligned}
 t=9 \quad RR_4 &= \frac{6+6}{6} = 2 \\
 RR_5 &= \frac{5+5}{5} = 2
 \end{aligned}$$

Priority Scheduling :-

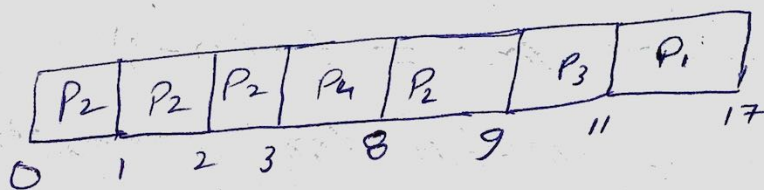
Process	BT	AT	(Preemptive)		(Non-Preemptive)		Non-Preemptive		
			Priority	ST	CT	RT	TAT	WT	
P ₁	6	0	1(L)	11	17	11	17	11	
P ₂	4	0	3	0	4	0	4	0	
P ₃	2	2	2	9	11	7	9	7	
P ₄	5	3	4(H)	4	9	1	6	1	
						<u>4.72</u>	<u>9</u>	<u>4.72</u>	Avg



$$CPU\% = \frac{17}{17} \times 100 = 100\%$$

$$Throughput = \frac{4}{17}$$

Preemptive



Preemptive

Process	BT	AT	Priority	ST	CT	(ST-AT)	(CT-AT)	(TAT-BT)	
						RT	TAT	WT	
P ₁	6	0	1(L)	11	17	11	17	11	
P ₂	4	0	3	0	9	0	9	5	
P ₃	2	2	2	9	11	7	9	7	
P ₄	5	3	4(H)	3	8	0	5	0	
						<u>4.5</u>	<u>10</u>	<u>5.72</u>	Avg.

$$CPU\% = \frac{17}{17} \times 100 = 100\%$$

$$Throughput = \frac{4}{17}$$

Round Robin (RR)

- Each process is given a time Quanta t_q .
- If it is completed before it then we schedule next process immediately.
- If it want to execute more then we will put it at the end of ready queue after t_q time quanta is over.
- RR is preemptive in nature & doesn't suffers from starvation.
- RR gives a bound on response time for a process.

$$\text{max response time} = t_q(n-1)$$

- RR makes it useful for interactive system.
- If t_q increases then context switch decreases & response time increases.
- If t_q decreases then context switch increases & response time decreases.
- If t_q is very large
RR $\rightarrow$ FCFS
- If t_q is very small CPU % is very less.
- If t_q is = C.S.T

$$\text{CPU \%} = \frac{t_2}{t_1 + t_2} \times 100 = \frac{1}{2} \times 100 = 50\%$$

Q. Which of the following statements are true.

1. SRTF may cause starvation. (TRUE)
2. Preemptive scheduling may cause starvation. (TRUE)
3. RR is better than FCFS in response time. (TRUE)

Q. Process	A.T.	B.T.
P ₁	0	5.520
P ₂	1	7.53
P ₃	3	4.20

Completion process of P₁, P₂ & P₃ under FCFS & RR₂

are:

- a) FCFS: P₁ P₂ P₃
 RR₂: P₁ P₂ P₃
- b) FCFS: P₁ P₂ P₃
 RR₂: P₁ P₃ P₂
- c) FCFS: P₁ P₂ P₃
 RR₂: P₁ P₃ P₂
- d) FCFS: P₁ P₃ P₂
 RR₂: P₁ P₂ P₃

Q. Consider 3 process 0, 1, 2 with bt of 2, 4, 8 all arrive at 0, using lotf (ties are broken with lowest pid first).

P	bt	at	ct	tat
0	2	0	12	12
1	4	0	13	13
2	8	0	18	18

$$\text{avg tat} = \frac{39}{3} = 13$$

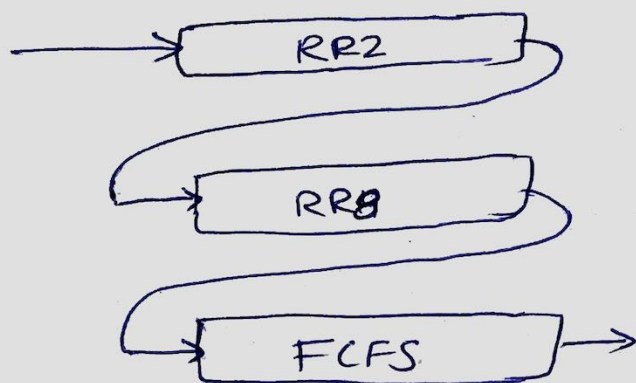
2	1	2	1	2	0	P ₁	P ₂	P ₀
---	---	---	---	---	---	----------------	----------------	----------------

0 4 5 6 7 8 9 10 11

P ₁	P ₂
----------------	----------------

12 13 14

Multilevel Feedback Queue :-



Process	A.T.	B.T.
P ₀	0 2 14	14 8 10
P ₁	14	6 4 0
P ₂	2 6 27	13 13 0
P ₃	3	10

P_{01}	P_{11}	P_{21}	P_{02}	P_{31}	P_{12}	P_{22}	P_{03}	P_{23}	
0	2	4	6	14	15	19	27	28	31

Prediction Techniques

Static

- ① Process size
- ② Process type

Dynamic

① Simple Averaging

- Given n - processes $(P_1, P_2, \dots, P_n)$
- t_i be the actual burst time & T_i denotes the predicted burst time.

$$T_{n+1} = \frac{1}{n} \sum_{i=1}^n t_i$$

$$\tau_2 = \alpha t_1 + (1-\alpha)\tau_1$$

$$\tau_3 = \alpha t_2 + (1-\alpha)\tau_2$$

$$= \alpha t_2 + (1-\alpha)[\alpha t_1 + (1-\alpha)\tau_1]$$

$$= \alpha t_2 + \alpha(1-\alpha)t_1 + (1-\alpha)^2\tau_1$$

② Exponential Averaging

$$\tau_{n+1} = \alpha t_n + (1-\alpha)\tau_n$$

t_n = actual CPU burst for n^{th} process

τ_n = Estimate of CPU burst for n^{th} time.

α = smoothening parameter

$$\tau_4 = \alpha t_3 + (1-\alpha)\tau_3$$

$$= \alpha t_3 + (1-\alpha)[\alpha t_2 + \alpha(1-\alpha)t_1 + (1-\alpha)^2\tau_1]$$

$$= \alpha t_3 + \alpha(1-\alpha)t_2 + \alpha(1-\alpha)^2t_1 + (1-\alpha)^3\tau_1$$

$$\tau_5 = \alpha t_4 + \alpha(1-\alpha)t_3 + \alpha(1-\alpha)^2t_2 + \alpha(1-\alpha)^3t_1 + (1-\alpha)^4\tau_1$$

eg:- $\alpha = 0.6$ $\tau_1 = 10$

actual burst time = 5, 9, 5 for t_1, t_2, t_3

respectively $\tau_4 = ?$

$$\tau_4 = \alpha t_3 + \alpha(1-\alpha)t_2 + \alpha(1-\alpha)^2t_1 + (1-\alpha)^3\tau_1$$

$$= 0.6(5) + 0.6(0.4)9 + 0.6(0.16)5 + (0.064)10$$

$$= 3 + 2.16 + 0.48 + 0.64$$

$$= 5.16 + 0.48 + 0.64$$

$$= 5.70 + 0.48$$

$$= 6.28$$

Batch OS

Advantages:

- Easy to handle repeated task.
- can be shared among multiple users.
- efficient for small

Disadvantages:

- Lot of time is wasted in reading & writing punch cards.
- Other jobs has to wait for unknown time if any job in execution is taking longer period.

Time Sharing OS : In this each process gets a specified time to execute on CPU. As there are multiple processes waiting to execute on a single CPU, specified time allotted to a process is known as time quanta (t_q)

Advantages:

- Fair chance to get CPU for every task.
- CPU utilization is high as CPU will be idle for less amount of time.

Disadvantages:

- Security & integrity of the processes is an overhead.
- Communication of shared data is to be managed.

Process Synchronization

P₁

for (i=0; i<n; i++)
count++;

P₂

for (i=0; i<n; i++)
count--;

-n to +n

~~I₁ MOV R₁, COUNT~~

~~I₂ INC R₁~~

~~I₃ INC R₁~~

~~I₄ MOV COUNT, R₁~~

~~I₅ MOV R₂, COUNT~~

~~I₆ DEC R₂~~

~~I₇ DEC R₂~~

~~I₈ MOV COUNT, R₂~~

~~I₁ I₂ I₃ I₄ I₅ I₆ I₇ I₈ I₉ I₁₀ I₁₁ I₁₂ I₁₃ I₁₄~~ → +2

~~I₁ I₂ I₃ I₄~~

count++
count++

1 MOV C, R₁
2 INC R₁
3 MOV R₁, C
4 MOV C, R₁
5 INC R₁
6 MOV R₁, C

count--
count--

7 MOV C, R₂
8 DEC R₂
9 MOV R₂, C
10 MOV C, R₂
11 DEC R₂
12 MOV R₂, C

I₁ I₅ I₇ to I₁₂ I₆

+2

I₇ to I₁₁ I₁ to I₆ I₁₂

-2

Critical section: A section of code where race condition may occur.

Race condition: When the order of execution decides output.
or

A condition when final result depends on interleaving of execution of process.

Critical section: A part of code which consist of one or more shared variable & improper ordering of execution may result in race condition.

Requirements of Process Synchronization:

① Mutual Exclusion: Only one process should access the shared variable at a time inside critical section.

② Process Requirement: No deadlock & no process should permanently bar the entry of start process to critical section.

③ Bounded waiting: No starvation & bound on waiting time.

④ Portability / architectural Neutral: No specific architecture or O.S. is concerned. Generic O.S. & architecture is considered while discussing all these facts.

Busy waiting :- When a Process continuously checking for critical section then CPU cycles gets wasted & process is in Busy waiting.

H.W

+1 $\Rightarrow$ 7, 8, 9, 1 to 5, 10, 11, 12, 6

-1 $\Rightarrow$ -1, 2, 3, 7 to 11, 4, 5, 6, 12

Process Synchronization

Software based Solⁿ

→ Algo1
 → Algo2 } wrong
 → Algo3
 → Peterson's Solⁿ
 or
 Becken's algo

O.S. Provided Structure

* Semaphore monitor
 wait signal

H/W based Solⁿ

TestAndSet
 Swap

Software Based Solⁿ

Algo ①

P1 shared int flag = 0
 while(flag == 0);
 C.S.
 flag = 1
 or flag = 0

P1
 Entry Code
 C.S.
 Exit Code

P2
 Entry Code
 C.S.
 Exit Code

P2
 while(flag == 1);
 C.S.

flag = 0 $\rightarrow$ can only execute 1
 or flag = 1 $\rightarrow$ can only execute alternate

Algo 2

shared int lock=0

P1

P2

- ① while (lock != 0);
- ② lock = 1
- ③ C.S.
- ④ lock = 0

- ⑤ while (lock != 0);
- ⑥ lock = 1
- ⑦ C.S.
- ⑧ lock = 0

1, 5, 2, 3, 6, 7 → violation of mutual exclusion.

Algo 3

P1

Shared int flag[2] = {0, 0}

P2

- ① flag[0] = 1;
- ② while (flag[1]);
- ③ C.S.
- ④ flag[0] = 1;

- ⑤ flag[1] = 1;
- ⑥ while (flag[0]);
- ⑦ C.S.
- ⑧ flag[1] = 0;

① → ② → ③ → ④ → ⑤ → ⑥ violation of process violation
① → ⑤ → ② → ⑥ violation of bounded waiting

Algo 4: Peterson's Soln / Bakery Algo
shared int turn = 0, flag[2] = {0, 0}

P1

P2

- 1 flag[0] = 1;
- 2 turn = 1;
- 3 while (flag[1] && turn == 1);
- 4 C.S.
- 5 flag[0] = 0;

- 6 flag[1] = 1;
- 7 turn = 0;
- 8 while (flag[0] && turn == 0);
- 9 C.S.
- 10 flag[1] = 0;

```

wait(semaphore mutex){ or down or P
    mutex--;
    if (mutex < 0){
        Block the calling process
        & add it to queue
    }
}

```

```

signal(semaphore mutex){ or up or V
    mutex++;
    if (mutex <= 0){
        wakeup the front process
        of the block process from
        the queue
    }
}

```

Shared semaphore $\text{mutex} = 0;$

P1

wait(mutex)

C.S.

signal(mutex)

deadlock

P2

wait(mutex)

C.S.

signal(mutex)

shared semaphore $\text{mutex} = 1;$

P1

$\text{wait}(\text{mutex})$

C.S.

$\text{signal}(\text{mutex})$

P2

$\text{wait}(\text{mutex})$

C.S.

$\text{signal}(\text{mutex})$

Synchronized

1. Producer - Consumer Problem
2. Reader - Writer Problem
3. Dining Philosopher Problem

① Producer Consumer Problem

$\text{mutex} = 1$

Producer

$\text{wait}(\text{mutex})$

produce

$\text{signal}(\text{mutex})$

Consumer

$\text{wait}(\text{mutex})$

consume

$\text{signal}(\text{mutex})$

$\Rightarrow$ Not Synchronized

$\text{mutex} = 1$

Producer

$\text{signal}(\text{filledspace})$

$\text{wait}(\text{mutex})$

produce

$\text{signal}(\text{mutex})$

~~$\text{wait}(\text{filledspace})$~~

$\text{filledspace} = 0$

Consumer

$\text{wait}(\text{filledspace})$

$\text{wait}(\text{mutex})$

consume

$\text{signal}(\text{mutex})$

$\text{signal}(\text{filledspace})$

$\Rightarrow$ Not Synchronized

emptySpace = 4
filledSpace = 0

mutex = 1

P_i

- ① wait(mutex)
- ② wait(emptySpace)
- ③ Produce
- ④ signal(mutex)
- ⑤ signal(filledSpace)

C_i

- ⑥ wait(mutex)
- ⑦ wait(filledSpace)
- ⑧ consume $\Rightarrow$ Not Synchronized
- ⑨ signal(mutex)
- ⑩ signal(emptySpace)

6, 7, 1 $\rightarrow$ deadlock

② Reader-Writer Problem

Reader

Writer

R $\rightarrow$ R $\checkmark$
R $\rightarrow$ W $\times$
W $\rightarrow$ R $\times$
W $\rightarrow$ W $\times$

Producer Consumer Problem

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int mutex = 1;
```

```
int x = 0;
```

```
void producer(){
```

```
    mutex--;
```

```
    x++;
```

```
    printf("\n Producer produced item %d", x);
```

```
    mutex++;
```

```
}
```

```
void consumer(){
```

```
    mutex--;
```

```
    if(x > 0){
```

```
        printf("\n Consumer consumes item %d", x);
```

```
        x--;
```

```
    }
```

```
    else{
```

```
        printf("\n Buffer is empty!");
```

```
    }
```

```
    mutex++;
```

```
}
```

```
int main(){
```

```
    int n;
```

```
    printf("\n1 Press 1 for Producer\n
```

```
        "\n2 Press 2 for Consumer\n
```

```
        "\n3 Press 3 for Exit");
```

```

for ( ; ; ) {
    printf("Enter choice : ");
    scanf("%d", &n);
    switch(n) {
        case 1:
            if (mutex == 1) {
                producer();
            }
            else {
                printf("In Unable to produce,
                    mutex is locked!");
            }
            break;
        case 2:
            if (mutex == 1) {
                consumer();
            }
            else {
                printf("In Unable to consume,
                    mutex is locked!");
            }
            break;
        case 3:
            exit(0);
            break;
    }
}
}

```

```
#include <iostream>
#include <queue>
#include <thread>
#include <mutex>
#include <condition_variable>
```

```
queue<int> buffer;
```

```
const unsigned int MAX_BUFFER_SIZE = 10;
```

```
mutex mtx;
```

```
condition_variable cv;
```

```
bool done = false;
```

```
bool notified = false;
```

```
void producer(int id) {
```

```
    int value = 0;
```

```
    while (value < 50) {
```

```
        unique_lock<mutex> lock(mtx);
```

```
        cv.wait(lock, [&] { return buffer.size() <
            MAX_BUFFER_SIZE; });
```

```
        buffer.push(value);
```

```
        cout << "Producer " << id << " produced: " <<
            value << endl;
```

```
        value++;
```

```
        notified = true;
```

```
        cv.notify_all();
```

```
    }
```

```
    done = true;
```

```
    cv.notify_all();
```

```
}
```

```

void consumer(int id) {
    while (true) {
        unique_lock<mutex> lock(mtx);
        cv.wait(lock, [&] { return !buffer.empty() || done; });

        if (!buffer.empty()) {
            int value = buffer.front();
            buffer.pop();
            cout << "Consumer " << id << " consumed: "
                 << value << endl;
        } else if (done) {
            break;
        }
        cv.notify-all();
    }
}

```

```

}
int main() {
    thread p1(producer, 1);
    thread p2(producer, 2);
    thread c1(consumer, 1);
    thread c2(consumer, 2);

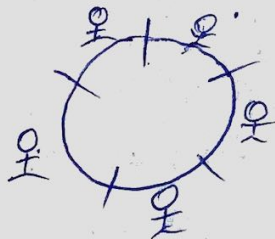
    p1.join();
    p2.join();
    c1.join();
    c2.join();
    return 0;
}

```

Reader Writer Problem

- | R | $r-c=0$
mutex=1 | W |
|------------------|--------------------|-----------------|
| ① $r-c++$ | | ⑧ wait(mutex) |
| ② if($r-c==1$) | | ⑨ c.s |
| ③ wait(mutex) | | ⑩ signal(mutex) |
| ④ Read | | |
| ⑤ $r-c--$ | | |
| ⑥ if($r-c==0$) | | |
| ⑦ signal(mutex) | | |
- ⇒ Not Synchronized

Dining Philosopher Problem



- P_i
- wait(chopstick[i] % 5)
 - wait(chopstick[(i+1) % 5])
 - eat
 - signal(chopstick[(i+1) % 5])
 - signal(chopstick[i] % 5)
- Not synchronized
but poor resource management

$m=1$

wait(m)

wait($i \% 5$)

wait($(i+1) \% 5$)

signal(m)

eat

signal($i \% 5$)

signal($(i+1) \% 5$)

~~Not~~ synchronized but
poor resource utilization

$m=1$

wait(m)

if($i \% 2 == 0$) {

wait($i \% 5$)

wait($(i+1) \% 5$)

}

else {

wait($(i+1) \% 5$)

wait($i \% 5$)

}

signal(m)

eat

signal($i \% 5$)

signal($(i+1) \% 5$)

Reader Writer

R

$r-c=0$

$m=1$

$w-c=1$

$r++$

$wait(m)$

$if(r-c==1)$

$wait(w-c)$

$signal(m)$

$read$

$wait(m)$

$r--$

$if(r-c==0)$

$signal(w-c)$

$signal(m)$

W

$wait(w-c)$

$write$

$signal(w-c)$

Hardware Based Solution

① TestAndSet

int lock = 0;

while(TestAndSet(lock));

C.S.

lock = 0;

{ int temp = lock;

lock = 1;

return temp;

}

② swap

 global local
swap(lock, lock1)
 " "
 0 1
 1 0

Swap(lock, lock1)

#Types of OS

Multiprocessing OS

- In multiprocessing OS, more than one CPU is present inside a single computer.
- Multiple processes can be executed at a time by assigning them to diff. CPUs.
- For eg: Dual core processor can process two processes at once & similarly quad core can process four processes at once.
- Sharing of bus, clock, memory & I/O devices will be done in multiprocessing OS.

Advantages:

- Faster speed of computer as multiple processes can be executed simultaneously.

Disadvantages:

- Data communication b/w processes is a difficult task.
- Data security is also an important issue.

Multiprogram OS.

In real scenario there are lot of processes waiting for execution i.e. waiting for CPU to be allocated while the main memory is not so large to accommodate all waiting processes so processes have to wait in job pool. CPU selects one of the processes kept in job pool using some job scheduling criteria & allocates it to CPU, then it starts executing. The process will

execute until it needs to do I/O or interrupted by some external factor like timer-interrupt, in that case another process can accommodate CPU immediately.

Real Time O.S.

Real Time OS are used in environments where a large no. of events mostly external to computer system must be accepted & processed in a short time and within certain deadline.

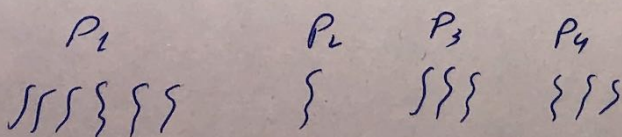
eg: metro control system, ATC, Telephone exchange equipment

Threads

{	{	}
Stack	S	S
PC	P	P
Reg	R	R
Code		
Data files		

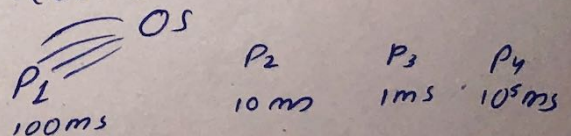
Threads (lightweight process)

User level thread
(O.S. does not know)



- Unfair process scheduling
- faster (no context switch)

Kernel level thread



- fair process scheduling
- slower (context switch)

Hybrid Thread

